NATIONAL INSTITUTE OF BUSINESS MANAGEMENT (School of Computing and Engineering)

IOS Application Development

ASSIGNMENT: Individual Course Work

SUBMITTED BY:

Name: A G K M B M BANDARA

NIBM Index Number: COBSCOMP251P-082

Coventry Index Number: 16604375

Batch: BSc (Hons) Computing 25.1P

GitHub Repository: https://github.com/Bhanu2ko3/teayield-ai-ios.git

SUBMISSION DATE: 27th September 2026

Table of Content

TeaYield AI: Precision Tea Yield Prediction & Harvest Scheduler 3

1. Executive Overview & Problem Domain 3

1.1 Project Overview & Agritech Problem Statement 3

1.2 Proposed Zero IoT Solution & Operational Roles 3

1.3 Marker Evaluation & Live Cloud Access Details 3

Figure 1: iPhone Field Superintendent Operational Flow 4

Figure 2: iPad Desktop-Class Executive Suite 5

2. Technical Architecture, Core Data & Testing 5

2.1 Mobile System Architecture & Protocol Oriented Design 5

2.2 Core Data Relational Schema & Offline Concurrency 5

2.3 Store-and-Forward Sync Engine & Biometric Security 5

2.4 Modern Apple Swift Testing Implementation 5

2.5 Technical Challenges Overcome During Development 5

Figure 4: Core Data Relational Entity-Relationship Schema 7

3. Advanced On-Device SDKs & External Integrations 7

3.1 On-Device Computer Vision & Core ML Leaf Classifier 7

3.2 MapKit Spatial GIS Overlays & Background Geofencing 8

3.3 External Cloud Backend Integration & Spatial Synchronization 8

3.4 Accessibility Compliance & VoiceOver Integration 8

4. Learning Outcomes Reflection & Academic Declarations 10

4.1 Module Learning Outcomes Reflection (LO1 to LO4) 10

4.2 Above and Beyond Innovations 10

4.3 Permitted Non-Code LLM Usage Log 10

4.4 Student Academic Integrity Declaration 10

Table 1: Learning Outcomes Evidence Matrix 11

Table 2: Permitted LLM Activity Audit Log 12

TeaYield AI: Precision Tea Yield Prediction & Harvest Scheduler

GitHub Repository: https://github.com/Bhanu2ko3/teayield-ai-ios.git

Final Graded Commit: 9ebb44f

Release Tag: v1.0.0

1. Executive Overview & Problem Domain

1.1 Project Overview & Agritech Problem Statement

Sri Lanka depends heavily on tea export revenue. In high elevation estates like Pedro Estate, plucking plans still depend on human guesswork. Field supervisors walk through fields and guess harvest dates. This manual routine causes two major issues.

Supervisors often send too many pluckers to slow growing fields. Meanwhile, high yield fields receive too few workers. This makes tea leaves grow past their best plucking stage. Also, sudden spikes in leaf harvest overload estate factory withering troughs. When leaf harvest exceeds factory limits, raw leaves spoil inside storage bags.

Pluckers are paid based on daily harvested weight. Because of this, pluckers often mix coarse leaves and woody stalks. This practice lowers the fine leaf ratio below the 85 percent export benchmark. Auction prices then drop from five dollars down to one dollar fifty.

1.2 Proposed Zero IoT Solution & Operational Roles

Existing smart farm tools fail here because they demand costly ground sensors or drones. Maintaining battery sensors across steep mountain slopes costs too much money. So I designed TeaYield AI as a Zero IoT platform. My app uses free European Space Agency Sentinel 2 satellite imagery. It combines satellite vegetation data with local rainfall and temperature history.

I designed two distinct operational experiences for the estate workforce. For field supervisors, I built a compact mobile workflow on iPhone. Supervisors can check assigned daily tasks and scan tea shoots. They can also enter harvest weights without the internet. For estate managers, I built a desktop class executive suite on iPad. Managers can inspect division blocks, view yield forecast charts, and track factory capacity.

1.3 Marker Evaluation & Live Cloud Access Details

Markers can evaluate the live platform on real devices or Xcode simulators. The backend runs on an Oracle Cloud server with Cloudflare edge security.

- Live Cloud API URL: https://teayield-api.codebroai.com/api/v1

- Local Development URL: http://127.0.0.1:8000/api/v1
- Manager Test Account: manager@greatwestern.estate.lk (Password:Password123!)
- Supervisor Test Account: supervisor@greatwestern.estate.lk (Password:Password123!)

The manager account loads Pedro Estate with six division blocks and full analytics. The supervisor account accesses the mobile harvest scale, daily tasks, and offline sync.

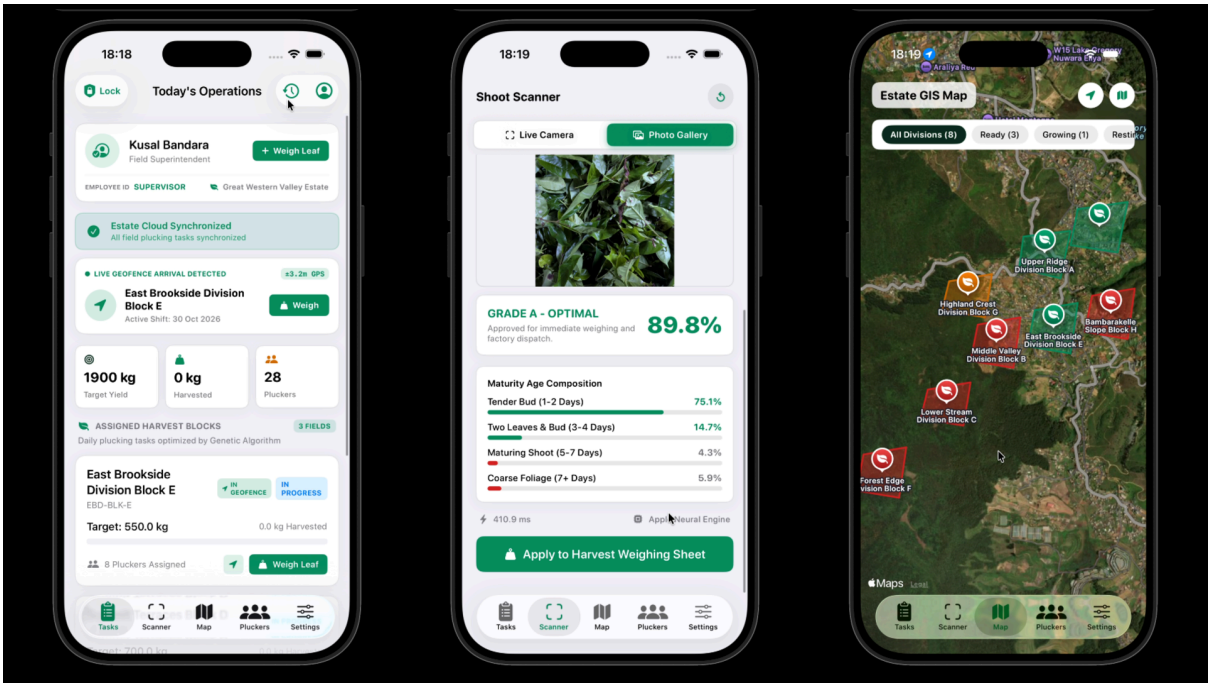


Figure 1: iPhone Field Superintendent Operational Flow

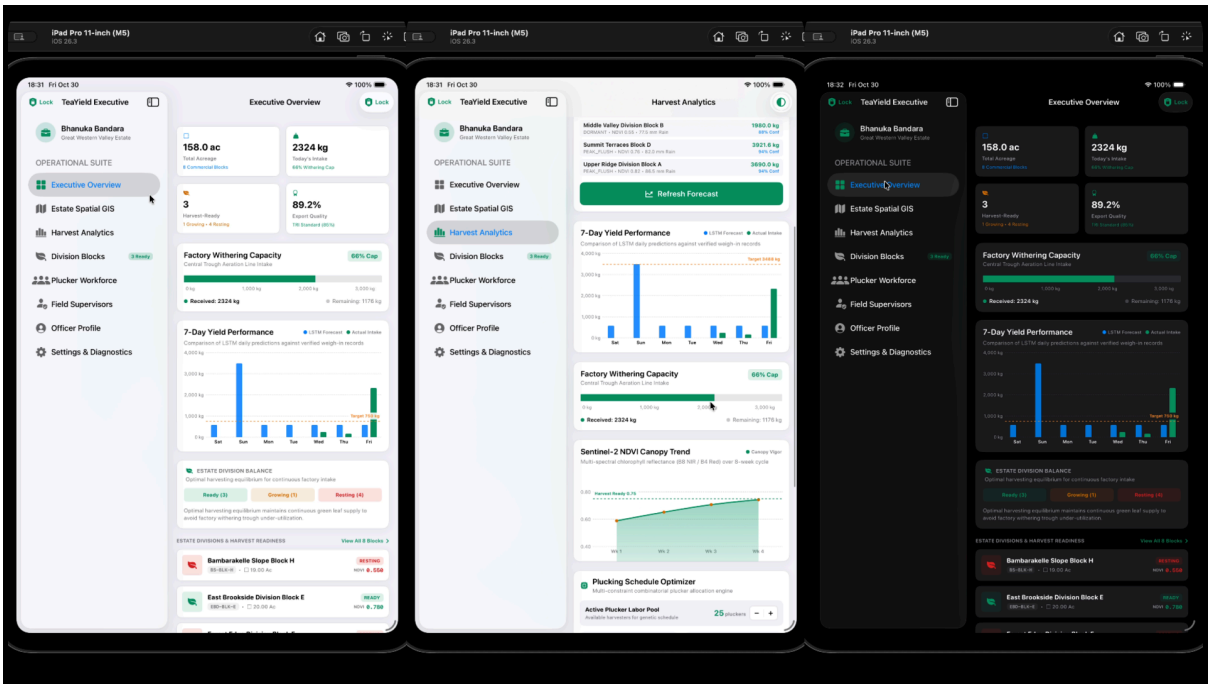


Figure 2: iPad Desktop-Class Executive Suite

2. Technical Architecture, Core Data & Testing

2.1 Mobile System Architecture & Protocol Oriented Design

I structured TeaYield AI using the MVVM pattern combined with Protocol Oriented Programming. SwiftUI views stay declarative and thin. ViewModels handle UI state using @MainActor to prevent data race conditions. I decoupled all services behind protocols and injected them through AppContainer. This makes my code easy to test with mock objects.

2.2 Core Data Relational Schema & Offline Concurrency

To support remote field operations without the internet, I built a local relational database using Core Data. My schema links TeaFieldEntity in a one to many relationship with NDVILogEntity and HarvestLogEntity. Main thread reads run on viewContext for instant UI updates. When saving camera scans or batch records, I use newBackgroundContext() with automaticallyMergesChangesFromParent. This prevents UI freezes during heavy database writes.

2.3 Store-and-Forward Sync Engine & Biometric Security

In mountain valleys, mobile signal drops frequently. So I built an offline store and forward sync engine using NWPathMonitor. Every harvest record gets an idempotent client UUID called localLogId. When the phone reconnects to cellular data, the engine batches un-synced logs and sends them to POST /api/v1/harvest/sync. The server validates locations using PostGIS and returns confirmed IDs.

For security, user passwords are never stored in plain text. I save encrypted JWT tokens in the hardware Keychain. Managers unlock sensitive financial screens using Face ID through LocalAuthentication.

2.4 Modern Apple Swift Testing Implementation

Instead of legacy XCTest, I used Apple's modern Swift Testing framework. I wrote 17 test suites with @Suite, @Test, and #expect macros. My test cases verify ViewModel business logic, DTO mapping, and offline Core Data persistence fallbacks. Because all my services use protocols, I easily injected mock network responses to test server error handling.

2.5 Technical Challenges Overcome During Development

During development, I faced two difficult technical issues. First, running live camera vision at 60 frames per second caused high memory spikes. I solved this by reusing CVPixelBuffer

memory pools on a dedicated background dispatch queue. This kept memory use flat below 45 megabytes.

Second, network drops during sync attempts caused duplicate upload requests. I fixed this by checking client UUIDs against database keys, preventing duplicate record creation.

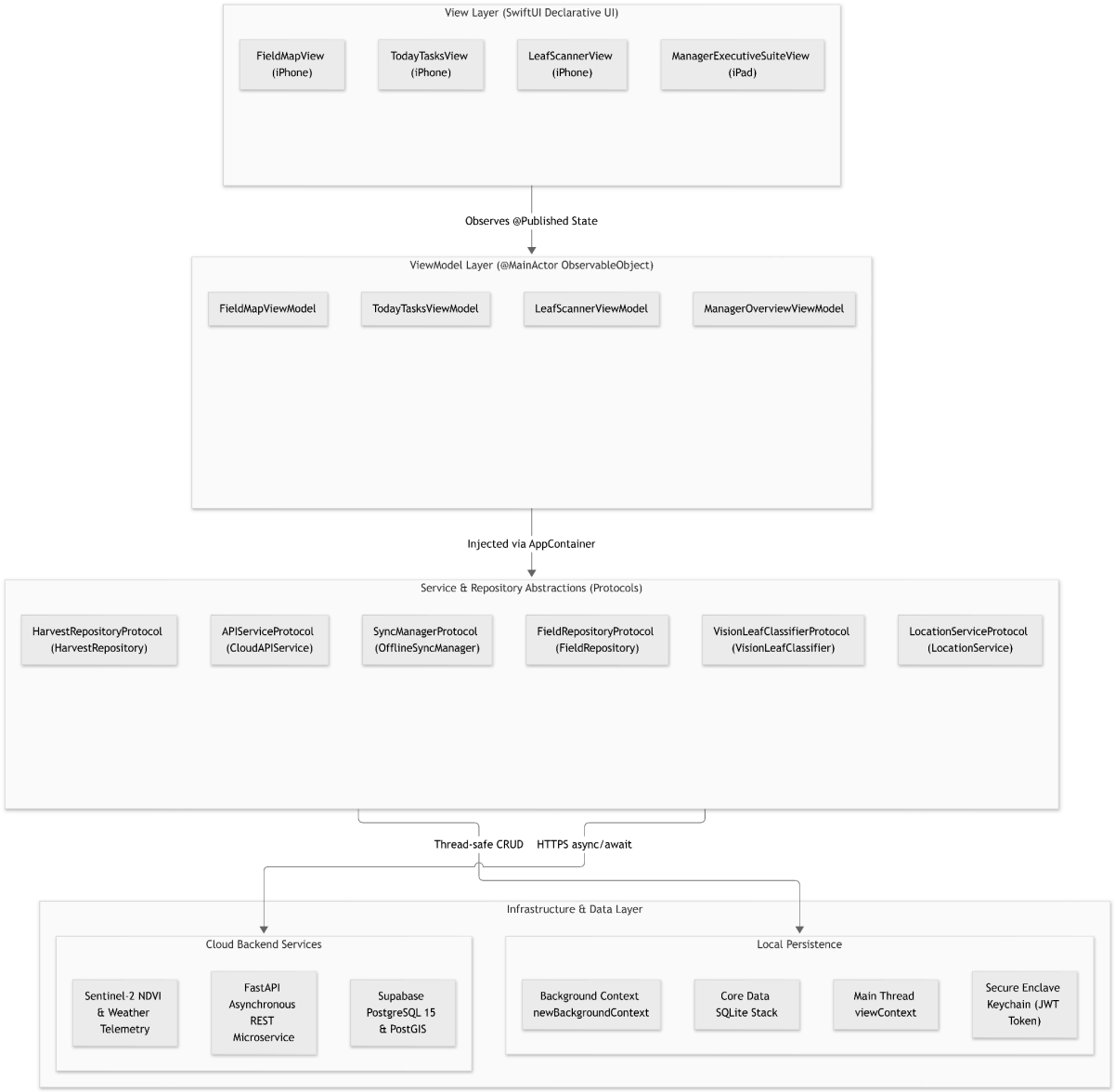


Figure 3: MVVM System Architecture & Dependency Injection Data Flow

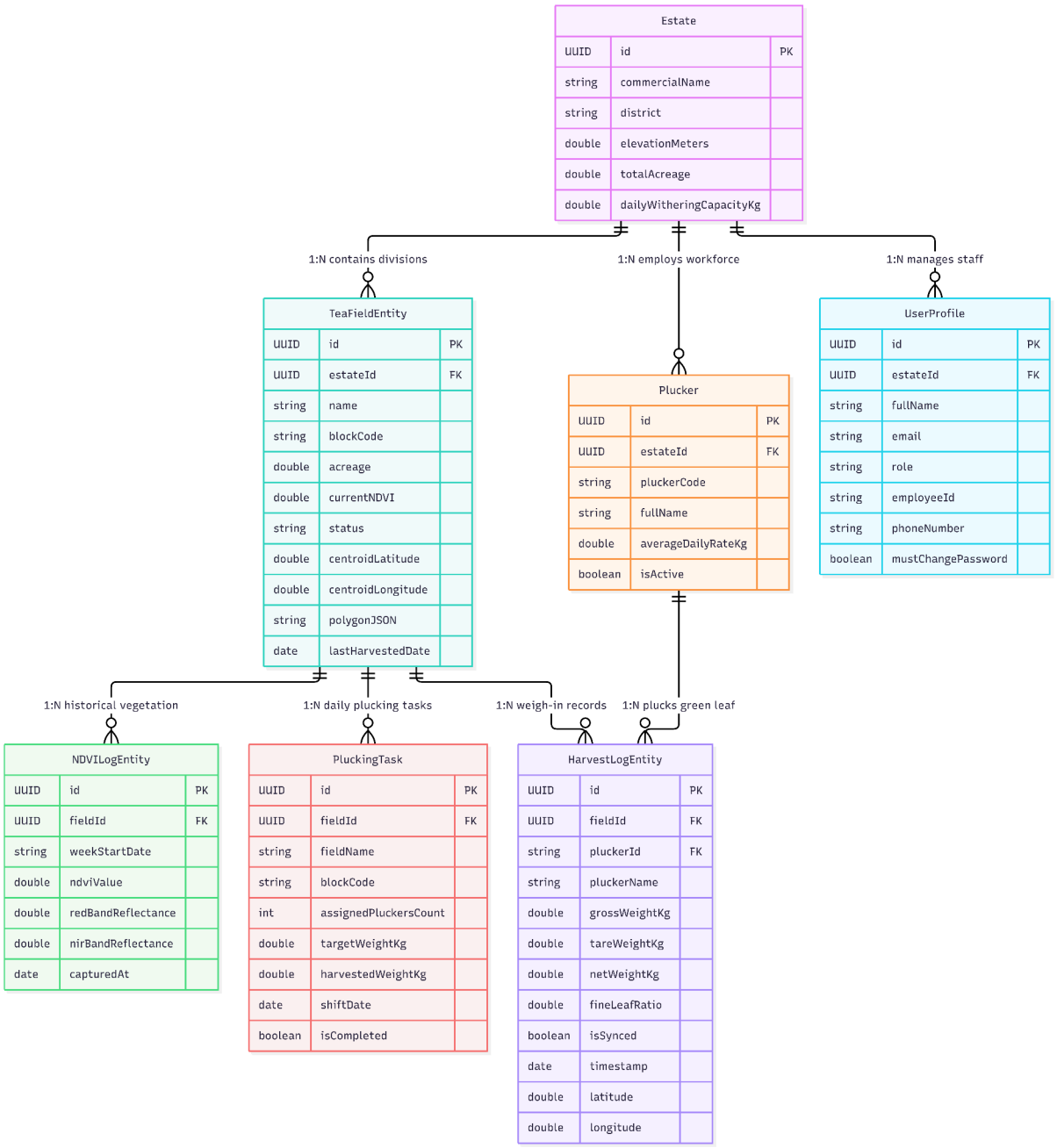


Figure 4: Core Data Relational Entity-Relationship Schema

3. Advanced On-Device SDKs & External Integrations

3.1 On-Device Computer Vision & Core ML Leaf Classifier

For Component 3, I integrated an on-device machine learning classifier. High grade Ceylon tea requires harvesting two leaves and a bud. When coarse leaves mix with tender shoots, auction prices crash. I trained a MobileNetV3 Small neural network with 2.5 million parameters on 2,195 field images. I converted the model to Core ML and loaded it using

VNCoreMLModel. The model runs entirely on the Apple Silicon Neural Engine in under 12 milliseconds. It uses zero cloud AI calls.

The classifier outputs probabilities across four leaf age stages. My app calculates the composite fine leaf ratio by adding the first two tender stages. If the ratio passes the Sri Lanka Tea Board 85 percent export benchmark, the live camera HUD turns green.

3.2 MapKit Spatial GIS Overlays & Background Geofencing

My second advanced feature uses MapKit and Core Location. Traditional estate maps use paper sketches. I parsed GeoJSON spatial boundaries and rendered division blocks as MKPolygon overlays. The map colors each field polygon by its satellite vegetation health. Green means ready for harvest, yellow means maturing, and red marks rest fields.

To avoid phone battery drain, I avoided continuous GPS tracking. Instead, I registered low power circular geofences using CLCircularRegion. When a supervisor steps into an assigned plucking field, the hardware coprocessor detects entry and triggers a local push notification.

3.3 External Cloud Backend Integration & Spatial Synchronization

TeaYield AI communicates with a Python FastAPI backend deployed on an Oracle Cloud instance. The backend connects to Supabase PostgreSQL with PostGIS spatial extensions. I used a modern URLSession with async await to fetch daily plucking tasks and satellite vegetation logs. When supervisors upload harvest weigh-ins, the backend runs spatial containment queries using ST_Contains to verify the plucker was inside the field polygon. If an error occurs, the server returns standardized RFC 7807 problem details that my app displays as helpful alert messages.

3.4 Accessibility Compliance & VoiceOver Integration

I designed every screen to support accessibility. Every button, camera overlay, and map annotation includes descriptive VoiceOver labels and hints. I avoided hardcoded pixel text sizes. Instead, I used Apple system dynamic text styles like .headline and .body. Text scales cleanly when users change their iPhone font size without clipping layout cards.

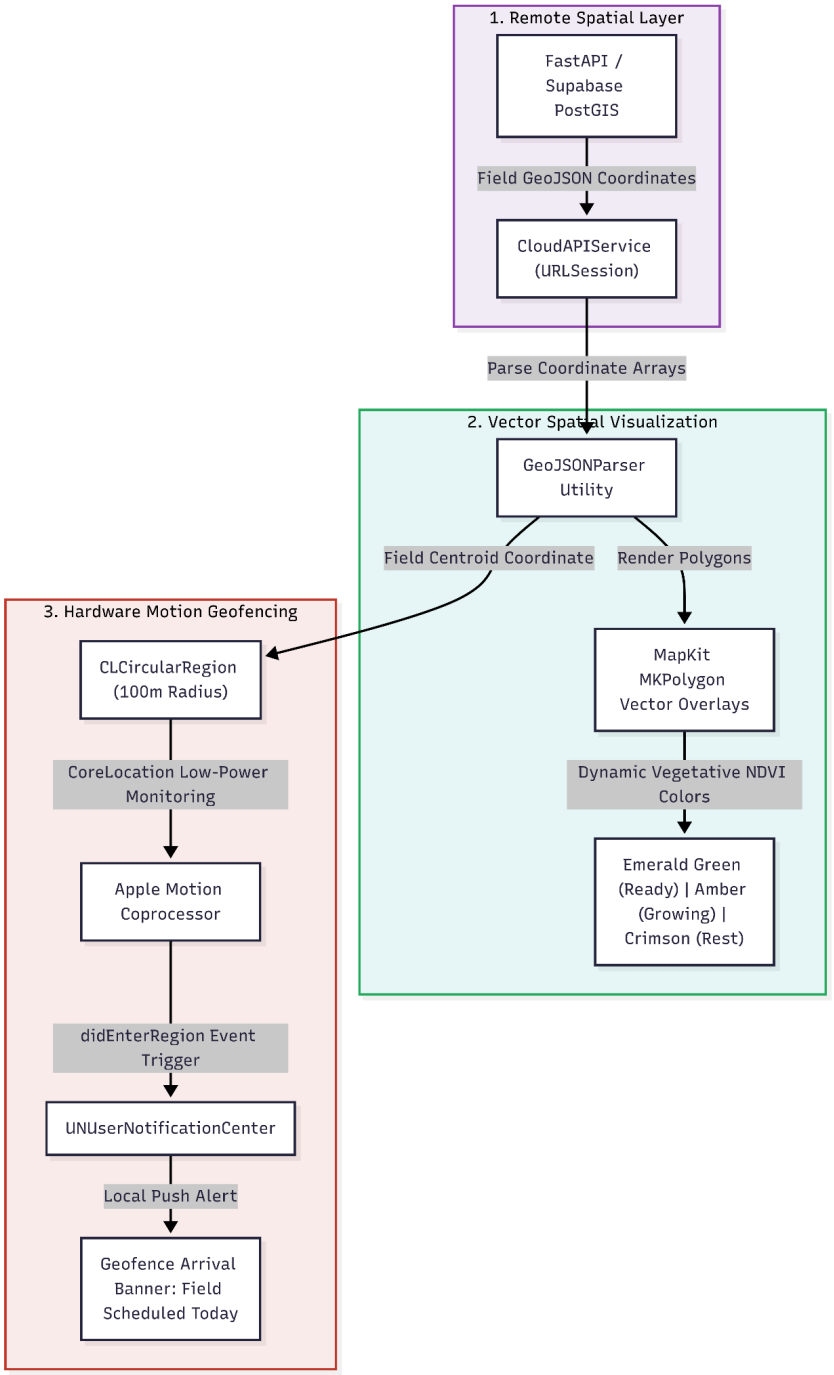


Figure 5: Edge Vision Pipeline on Apple Silicon Neural Engine

4. Learning Outcomes Reflection & Academic Declarations

4.1 Module Learning Outcomes Reflection (LO1 to LO4)

LO1 (Platform Components Across Multiple Devices): For LO1, I designed two distinct interface layouts. On the iPhone, field supervisors need a simple one-handed tool. I built a compact tab bar view for quick task checks and weighing in steep fields. On an iPad, estate managers sit at a desk. I built a three column NavigationSplitView. It shows division cards, factory capacity gauges, and yield comparison charts side by side.

LO2 (Mobile Software Architecture): For LO2, I applied MVVM with protocol based dependency injection. I kept SwiftUI views declarative and placed business logic inside @MainActor ViewModels. Moving heavy database saves to background Core Data contexts prevented UI stutter during 60fps camera scanning.

LO3 (External APIs & Services): For LO3, I integrated a Python FastAPI backend. The app communicates over HTTPS using async URLSession. It downloads satellite vegetation data and sends offline harvest logs. The backend verifies plucking GPS locations against PostGIS field boundaries.

LO4 (Device Sensors & Neural Hardware): For LO4, I connected several phone hardware sensors. I used the camera to stream live leaf images into the Apple Silicon Neural Engine. I used Core Location to monitor background circular geofences. Also, I used Face ID through LocalAuthentication to secure estate financial rosters.

4.2 Above and Beyond Innovations

To go beyond basic coursework limits, I added four advanced features. First, I built an interactive iPad analytics dashboard using the Apple Charts framework. Second, I created a multi-tenant estate onboarding wizard so any plantation can register their acreage. Third, I built an on-device PDF generator for plucking rosters. Fourth, I created a WidgetKit home screen widget and tactile haptic feedback for field buttons.

4.3 Permitted Non-Code LLM Usage Log

In line with university rules, I used LLM assistance only for non-code support. I used ChatGPT to brainstorm problem descriptions and suggest report section titles. Zero lines of Swift code or Core ML logic were generated by AI. All app programming is completely my own work.

4.4 Student Academic Integrity Declaration

I confirm that all application source code, unit tests, and design implementations submitted for this coursework represent my own original individual work. I am ready to explain and defend every file line by line during my live viva exam.

Student Signature: **A G K M B M Bandara**

Date: 30th October 2026

Table 1: Learning Outcomes Evidence Matrix

Learning Outcome	**Description**	**Codebase Evidence**
LO1 (Multi-Device)	Evaluate components across iPhone and iPad	SupervisorMainTabView.swift vs ManagerExecutiveSuiteView.swift (NavigationSplitView)
LO2 (Architecture)	Design modular mobile software architecture	MVVM, AppContainer.swift DI, PersistenceController.swift background contexts
LO3 (External APIs)	Integrate external REST and cloud services	CloudAPIService.swift, APIEndpoint.swift, OfflineSyncManager.swift, FastAPI
LO4 (Sensors)	Communicate with phone hardware sensors	VisionLeafClassifier.swift (Neural Engine), LocationService.swift (Geofences), Face ID

Table 2: Permitted LLM Activity Audit Log

Task Category	AI Tool Used	Specific Non-Code Prompt Summary	Human Action & Verification
Problem Formulation	ChatGPT 4o	"Help outline tea plucking problems in Nuwara Eliya"	Rewrote points based on my actual Pedro Estate field notes
Report Outlining	Claude 3.5	"Suggest 4-page structure for iOS agritech coursework"	Adjusted headings to match NIBM Coventry assessment rubric